

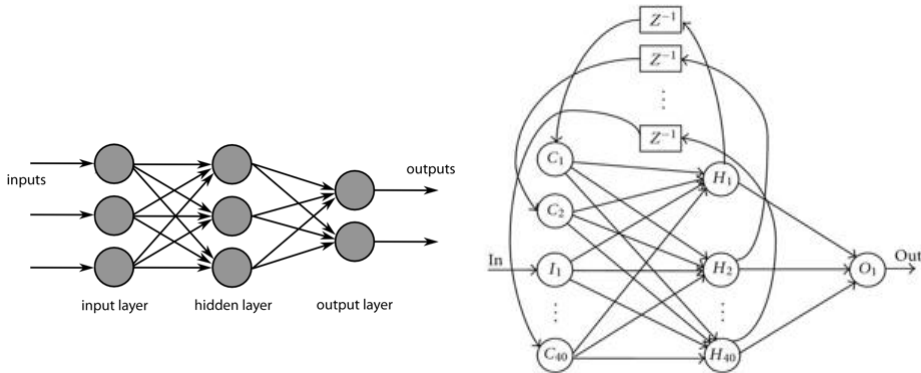
Recurrent neural networks

- ▶ Tüm problemler **sabit uzunlukta girişler ve çıkışlar ile ifade edilemez.**
- ▶ Konuşma tanıma, zaman serileri ve tahmin gibi problemler bilgiyi saklama ve kullanmayı gerektirir.
- ▶ Örneğin, girişteki bit dizisinde **1 sayısı çift ise EVET, tek ise HAYIR çıkışı** üreten sistemde önceki bilgiyi saklamak gerekir (1000010101 -> YES, 100011 -> NO).
- ▶ Bazı problemlerde sabit uzunlukta giriş her zaman mümkün olmayabilir ve giriş boyutu öncekilerden farklı olabilir.
- ▶ Tekrarlayan sinir ağları (**recurrent neural networks**) **önceki çıkışı** veya **gizli katmanın önceki durumlarını** giriş olarak alır.
- ▶ Herhangi bir t zamanındaki **giriş, geçmiş bilgisi ile anlık girişin birleşimidir.**

3

Recurrent neural networks

- ▶ Klasik yapay sinir ağlarında **önceki durumlar veya girişler arasında ilişkilendirme yapılmaz.**
- ▶ RNN'lerde **önceki girişler veya durumlar ile ilişkilendirme yapılır.**



4

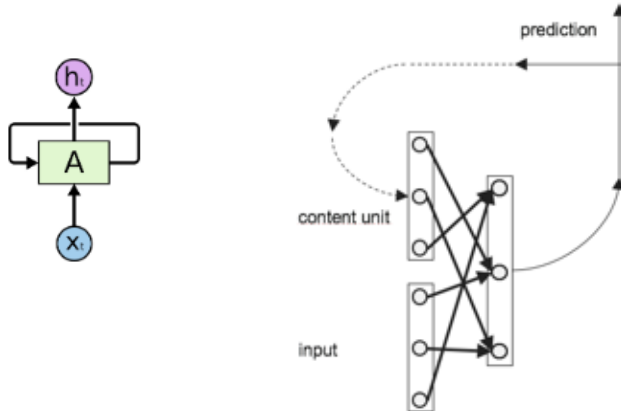
İçerik

- ▶ Recurrent neural networks
- ▶ RNN'lerin yapısı
- ▶ RNN'lerde ileri geçiş
- ▶ RNN eğitimi
- ▶ RNN uygulamaları

5

RNN'lerin yapısı

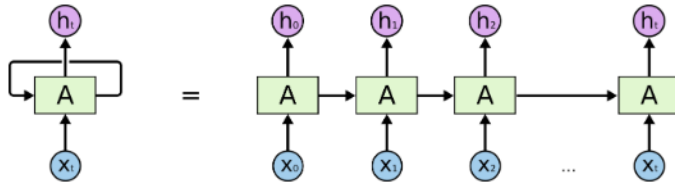
- ▶ RNN'ler döngüye sahiptir.
- ▶ Şekilde, **A** bir sinir ağını, x_t girişleri ve h_t ise çıkışı göstermektedir.



6

RNN'lerin yapısı

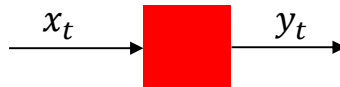
- ▶ Bir RNN, çok sayıda sinir ağı kopyası olarak düşünülebilir.
- ▶ Her sinir ağı bilgiyi sonrakine (girişe) aktarır.



7

RNN'lerin yapısı

- ▶ Basit ileri beslemeli ağlarda **her çıkış kendi girişi için hesaplanır.**



$$y_0 = f(W_x X_0)$$

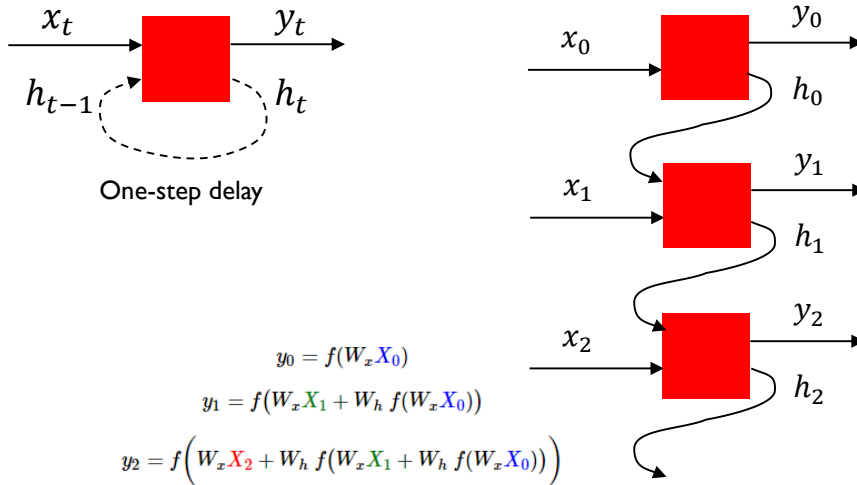
$$y_1 = f(W_x X_1)$$

$$y_2 = f(W_x X_2)$$

8

RNN'lerin yapısı

- ▶ RNN'lerde **her çıkış, kendi girişi ve önceki girişe bağlı olarak hesaplanır.**



9

RNN'lerin yapısı

- ▶ Her zaman aralığında **aynı fonksiyon** ve **aynı parametreler** kullanılır.
- ▶ **Katmanlar arasında ağırlıklar paylaşılarak kullanılır.**

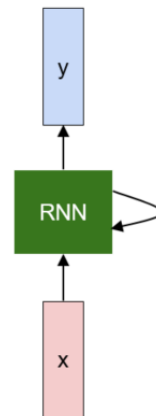
$$h_t = f_W(h_{t-1}, x_t)$$

new state

old state

input vector at some time step

some function with parameters W



10

RNN'lerin yapısı

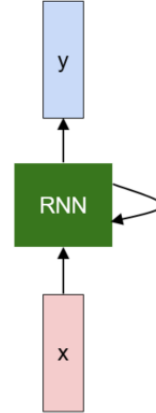
- ▶ RNN'lerde **önceki durum bilgisi belirli bir ağırlıkta sonraki çıkışları etkiler.**

$$h_t = f_W(h_{t-1}, x_t)$$



$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$$

$$y_t = W_{hy}h_t$$

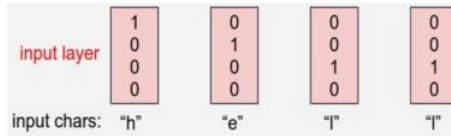


11

RNN'lerin yapısı

Örnek

- ▶ Sözlükte 4 harf olsun {h, e, l, o}.
- ▶ "hello" kelimesi için RNN oluşturalım.
- ▶ Giriş için **harfler vektöre dönüştürülür.**
- ▶ Kelimedeki her **harfin kendi sırası için 1, diğerleri 0** olacak şekilde giriş vektörleri oluşturulur.



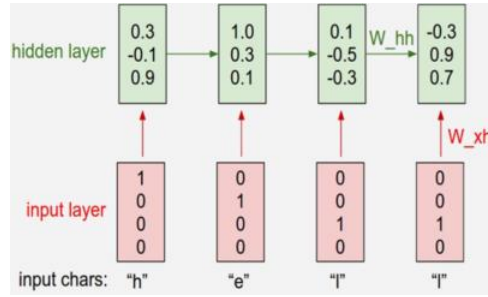
12

RNN'lerin yapısı

Örnek - devam

- Gizli katman çıkışları transfer fonksiyonu ile hesaplanır.

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

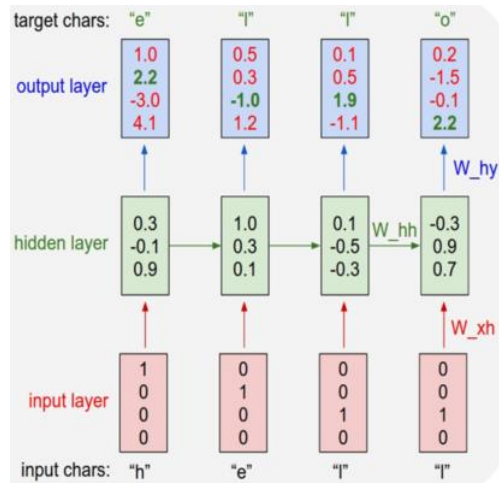


13

RNN'lerin yapısı

Örnek - devam

- Çıkışta hedeflenen vektöre göre hata hesaplanır.
- "h" karakteri girildikten sonra sonrakinin "e" olma olasılığı.
- "e" karakteri girildikten sonra sonrakinin "i" olma olasılığı.
- "i" karakteri girildikten sonra sonrakinin "i" olma olasılığı.
- "i" karakteri girildikten sonra sonrakinin "o" olma olasılığı.

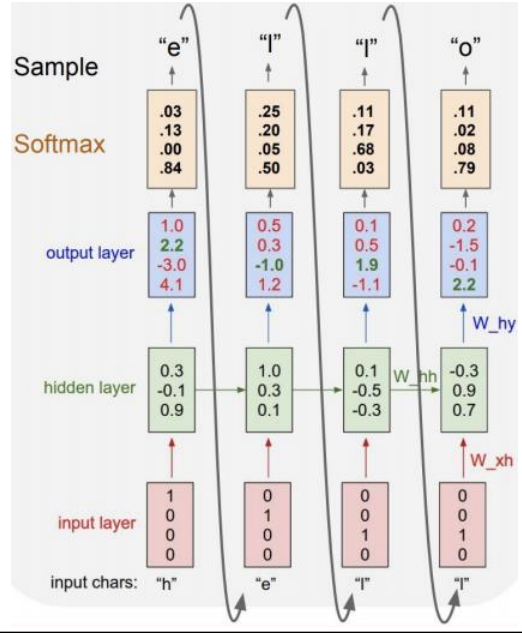


14

RNN'lerin yapısı

Örnek - devam

- Çıktılar girişe aktarılarak bir kelime/cümle oluşturulabilir.

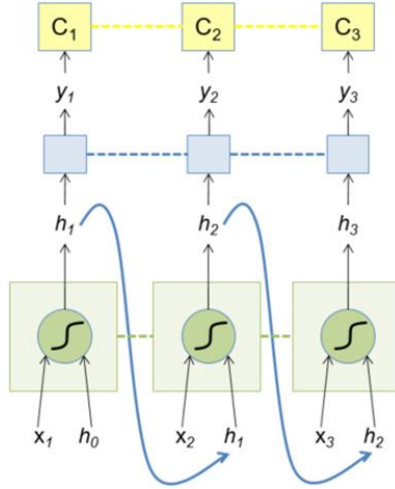


İçerik

- Recurrent neural networks
- RNN'lerin yapısı
- RNN'lerde ileri geçiş
- RNN eğitimi
- RNN uygulamaları

RNN'lerde ileri geiř

- Önceki ıkıř sonraki giriř ile birleřtirilerek yeni ıkıř hesaplanır.



$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

$$y_t = F(h_t)$$

$$C_t = \text{Loss}(y_t, T_t)$$

----- indicates shared weights

17

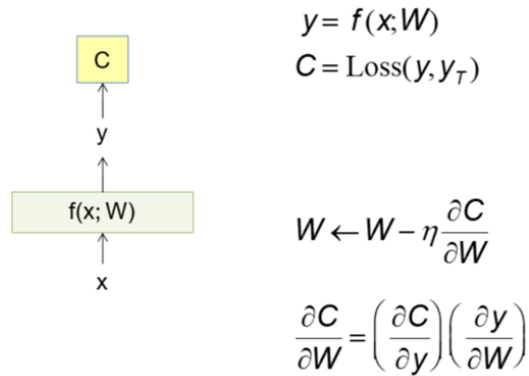
İerik

- Recurrent neural networks
- RNN'lerin yapısı
- RNN'lerde ileri geiř
- RNN eęitimi
- RNN uygulamaları

18

RNN eğitimi

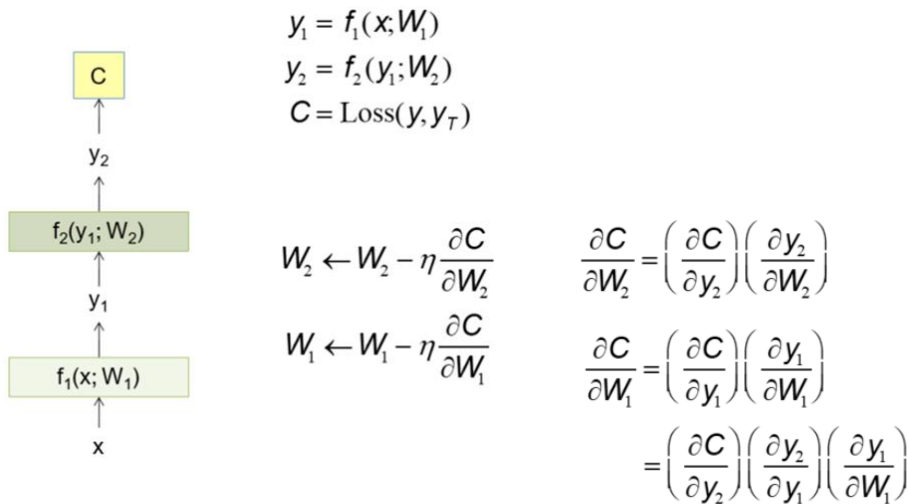
- ▶ RNN'ler için eğitim backpropagation through time (BPTT) ile yapılır.
- ▶ **İstenen çıkıştaki hataya göre ağırlıklar değiştirilir.**



19

RNN eğitimi

- ▶ **Çok katmanlı yapılarda** da geri yayılım yapılarak **ağırlıklar değiştirilir.**



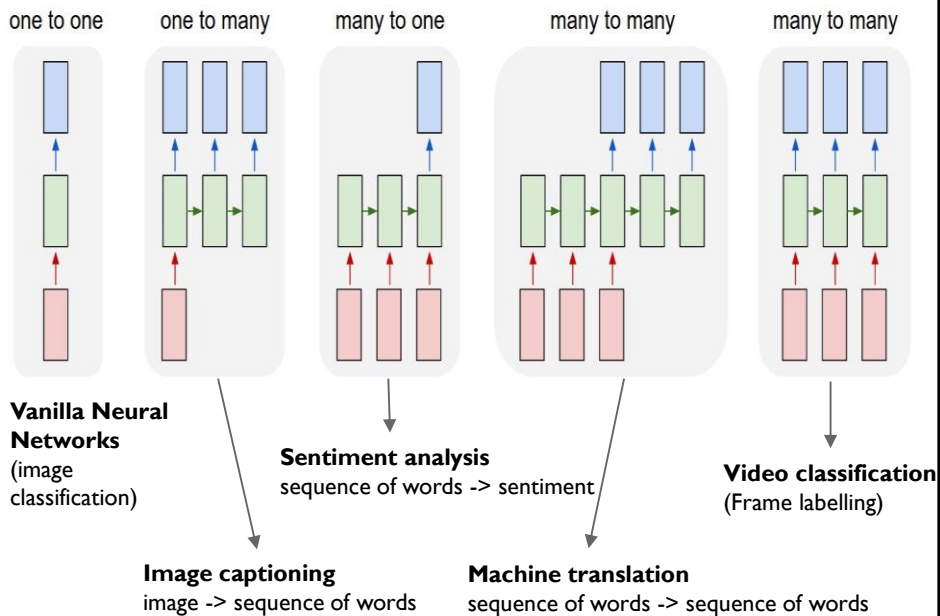
20

İçerik

- ▶ Recurrent neural networks
- ▶ RNN'lerin yapısı
- ▶ RNN'lerde ileri geçiş
- ▶ RNN eğitimi
- ▶ RNN uygulamaları

25

RNN uygulamaları

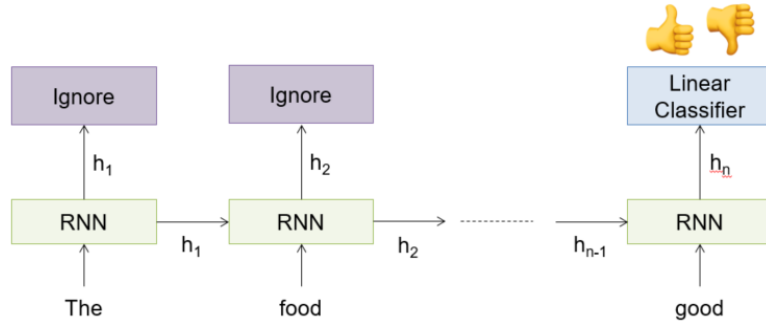


26

RNN uygulamaları

Duygu Sınıflandırma

- Çok sayıda cümle ile eğitilir.
- Daha sonra girilen cümleler için duygu durum sınıflandırması yapılır.
- Tek çıkış alınıp diğerleri göz ardı edilebilir.

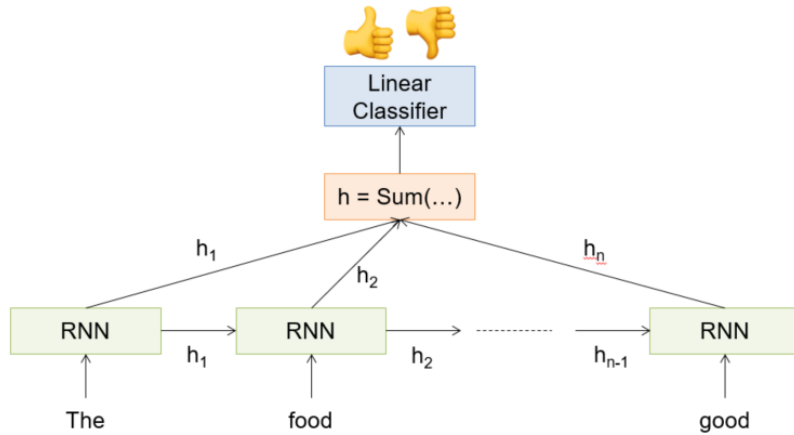


27

RNN uygulamaları

Duygu Sınıflandırma

- Tüm çıkışların toplamı da birleştirilerek kullanılabilir.



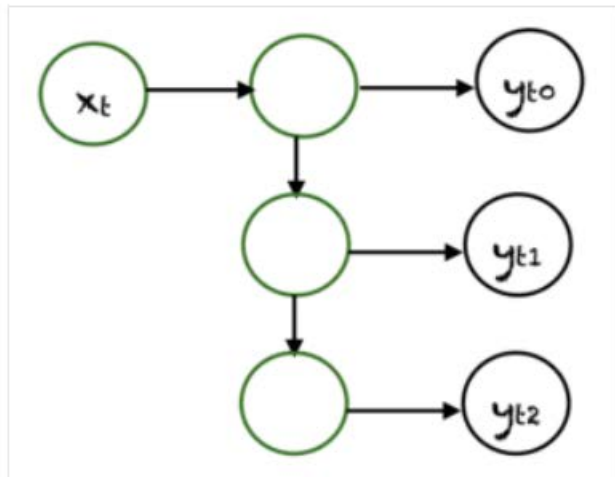
28

One to One



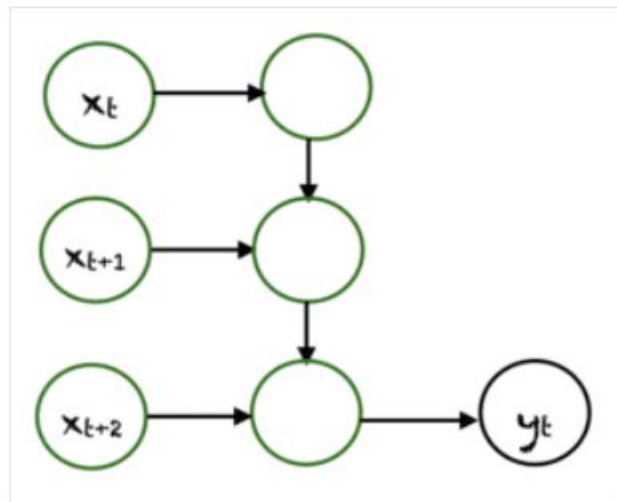
Here, there is a single (x_t, y_t) pair. Traditional neural networks employ a one-to-one architecture.

One to Many



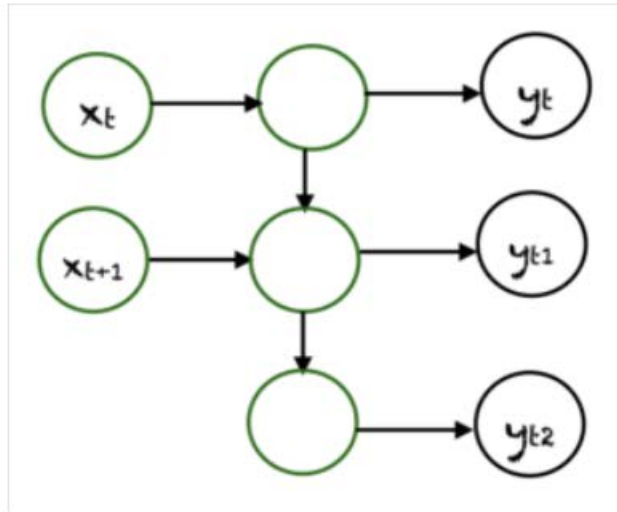
In one-to-many networks, a single input at x_t can produce multiple outputs, e.g., (y_{t0}, y_{t1}, y_{t2}) . Music generation is an example area where one-to-many networks are employed.

Many to One



In this case, many inputs from different time steps produce a single output. For example, (x_t, x_{t+1}, x_{t+2}) can produce a single output y_t . Such networks are employed in sentiment analysis or emotion detection, where the class label depends upon a sequence of words.

Many to Many

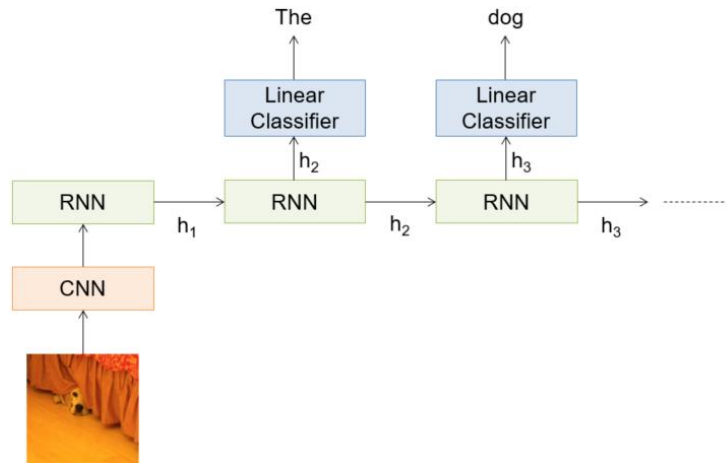


There are many possibilities for many-to-many. An example is shown above, where two inputs produce three outputs. Many-to-many networks are applied in machine translation, e.g., English to French or vice versa translation systems.

RNN uygulamaları

Image Captioning

- CNN ile birlikte görüntü etiketleme uygulamalarında kullanılmaktadır.



29

RNN uygulamaları

Image Captioning

- RNN ile görüntü etiketleme uygulamaları.



30